

Principles of Programming Languages

Declarative programming: functional programming

Andrei Arusoaie¹

¹Department of Computer Science

January 9, 2020

Outline

Paradigms

Outline

Paradigms

Declarative Programming

- Functional Programming

Outline

Paradigms

Declarative Programming
Functional Programming

Conclusion

Paradigms

- ▶ Imperative Programming: ✓

Paradigms

- ▶ Imperative Programming: ✓
- ▶ Object Oriented Programming: ✓

Paradigms

- ▶ Imperative Programming: ✓
- ▶ Object Oriented Programming: ✓
- ▶ Declarative Programming
 - ▶ Functional Programming
 - ▶ Logic Programming

Outline

Paradigms

Declarative Programming
Functional Programming

Conclusion

General facts about PLs - I

General facts about PLs - I

- ▶ Computational model based on:
state + modifiable variable + assignment

General facts about PLs - I

- ▶ Computational model based on:
state + modifiable variable + assignment
- ▶ *Computation proceeds by modifying values stored in locations*

General facts about PLs - I

- ▶ Computational model based on:
state + modifiable variable + assignment
- ▶ *Computation proceeds by modifying values stored in locations*
- ▶ The *von Neumann Machine*

General facts about PLs - I

- ▶ Computational model based on:
state + modifiable variable + assignment
- ▶ *Computation proceeds by modifying values stored in locations*
- ▶ The *von Neumann Machine*
- ▶ Conventional PL are based on this computational model

General facts about PLs - II

- ▶ However, this is not the only possible model upon which to base a PL

General facts about PLs - II

- ▶ However, this is not the only possible model upon which to base a PL:
 - ▶ It is possible to compute without referring to a state

General facts about PLs - II

- ▶ However, this is not the only possible model upon which to base a PL:
 - ▶ It is possible to compute without referring to a state. How?

General facts about PLs - II

- ▶ However, this is not the only possible model upon which to base a PL:
 - ▶ It is possible to compute without referring to a state. How?
 - ▶ ... by rewriting expressions

General facts about PLs - II

- ▶ However, this is not the only possible model upon which to base a PL:
 - ▶ It is possible to compute without referring to a state. How?
 - ▶ ... by rewriting expressions
 - ▶ The computation is expressed in terms of modifications of the environment

General facts about PLs - II

- ▶ However, this is not the only possible model upon which to base a PL:
 - ▶ It is possible to compute without referring to a state. How?
 - ▶ ... by rewriting expressions
 - ▶ The computation is expressed in terms of modifications of the environment
 - ▶ “Pure” principles: do not modify the ‘state’; no side-effects

Functional programming: history + characteristics

- ▶ This paradigm is as old as the imperative one

Functional programming: history + characteristics

- ▶ This paradigm is as old as the imperative one
- ▶ Turing machine vs. λ -calculus

Functional programming: history + characteristics

- ▶ This paradigm is as old as the imperative one
- ▶ Turing machine vs. λ -calculus
- ▶ Ingredients: higher-order functions + recursion

Functional programming: history + characteristics

- ▶ This paradigm is as old as the imperative one
- ▶ Turing machine vs. λ -calculus
- ▶ Ingredients: higher-order functions + recursion
- ▶ PLs: Haskell, Miranda (pure) + **Lisp**, ML, Scheme, ...

Some mathematical background

- ▶ Math function: $f(x) = x^2$

Some mathematical background

- ▶ Math function: $f(x) = x^2$, where $f : \mathbb{R} \rightarrow \mathbb{R}$

Some mathematical background

- ▶ Math function: $f(x) = x^2$, where $f : \mathbb{R} \rightarrow \mathbb{R}$
- ▶ Math function application: $f(2)$, with result 4

Some mathematical background

- ▶ Math function: $f(x) = x^2$, where $f : \mathbb{R} \rightarrow \mathbb{R}$
- ▶ Math function application: $f(2)$, with result 4
- ▶ But mathematicians say that they *define* $f(x)$

Some mathematical background

- ▶ Math function: $f(x) = x^2$, where $f : \mathbb{R} \rightarrow \mathbb{R}$
- ▶ Math function application: $f(2)$, with result 4
- ▶ But mathematicians say that they *define* $f(x)$
- ▶ Indeed the *name* of the function is f

Some mathematical background

- ▶ Math function: $f(x) = x^2$, where $f : \mathbb{R} \rightarrow \mathbb{R}$
- ▶ Math function application: $f(2)$, with result 4
- ▶ But mathematicians say that they *define* $f(x)$
- ▶ Indeed the *name* of the function is f
- ▶ ...but we distinguish the *definition* from the *application*

Some mathematical background

- ▶ Math function: $f(x) = x^2$, where $f : \mathbb{R} \rightarrow \mathbb{R}$
- ▶ Math function application: $f(2)$, with result 4
- ▶ But mathematicians say that they *define* $f(x)$
- ▶ Indeed the *name* of the function is f
- ▶ ...but we distinguish the *definition* from the *application*, respectively:
 - ▶ f has a *formal* parameter x which indicates the *transformation* that f applies to arguments x

Some mathematical background

- ▶ Math function: $f(x) = x^2$, where $f : \mathbb{R} \rightarrow \mathbb{R}$
- ▶ Math function application: $f(2)$, with result 4
- ▶ But mathematicians say that they *define* $f(x)$
- ▶ Indeed the *name* of the function is f
- ▶ ...but we distinguish the *definition* from the *application*, respectively:
 - ▶ f has a *formal* parameter x which indicates the *transformation* that f applies to arguments x
 - ▶ $f(x)$ is the expression that results after the transformation happens

Some mathematical background

- ▶ Math function: $f(x) = x^2$, where $f : \mathbb{R} \rightarrow \mathbb{R}$
- ▶ Math function application: $f(2)$, with result 4
- ▶ But mathematicians say that they *define* $f(x)$
- ▶ Indeed the *name* of the function is f
- ▶ ...but we distinguish the *definition* from the *application*, respectively:
 - ▶ f has a *formal* parameter x which indicates the *transformation* that f applies to arguments x
 - ▶ $f(x)$ is the expression that results after the transformation happens
- ▶ Functions = *expressible* values

More about writing function application

- ▶ Classical: $f(2)$

More about writing function application

- ▶ Classical: $f(2)$
- ▶ Available: $(f\ 2)$ or simply $f\ 2$

More about writing function application

- ▶ Classical: $f(2)$
- ▶ Available: $(f\ 2)$ or simply $f\ 2$
- ▶ The latter notation has an advantage:

More about writing function application

- ▶ Classical: $f(2)$
- ▶ Available: $(f\ 2)$ or simply $f\ 2$
- ▶ The latter notation has an advantage:
 - ▶ Anonymous functions: $\backslash x \rightarrow x * x$ – Haskell syntax

More about writing function application

- ▶ Classical: $f(2)$
- ▶ Available: $(f\ 2)$ or simply $f\ 2$
- ▶ The latter notation has an advantage:
 - ▶ Anonymous functions: $\backslash x \rightarrow x * x$ – Haskell syntax
 - ▶ Application: $(\backslash x \rightarrow x * x)\ 2$

More about writing function application

- ▶ Classical: $f(2)$
- ▶ Available: $(f\ 2)$ or simply $f\ 2$
- ▶ The latter notation has an advantage:
 - ▶ Anonymous functions: $\backslash x \rightarrow x * x$ – Haskell syntax
 - ▶ Application: $(\backslash x \rightarrow x * x)\ 2$
 - ▶ Higher-order functions, currying, ...

λ -calculus

- ▶ Syntax: $e ::= x \mid (e e) \mid \lambda x . e$
 - ▶ x is a non-terminal symbol
 - ▶ $(e e)$ - application
 - ▶ $\lambda x . e$ - abstraction
- ▶ Examples: 10 , $\lambda x . x$, $((\lambda x . x) 10)$, $(\lambda x . x + y)$

Binders

- ▶ $\lambda x . e$
 - ▶ The abstraction operator binds x
 - ▶ The renaming of the x does not change the semantics of the expression
 - ▶ Substitutions do not have effects over bound variables

Binders

- ▶ $\lambda x . e$
 - ▶ The abstraction operator binds x
 - ▶ The renaming of the x does not change the semantics of the expression
 - ▶ Substitutions do not have effects over bound variables

Free variables (Fv)	Bound variables (Bv)
$Fv(x) = \{x\}$	$Bv(x) = \emptyset$
$Fv(e_1 e_2) = Fv(e_1) \cup Fv(e_2)$	$Bv(e_1 e_2) = Bv(e_1) \cup Bv(e_2)$
$Fv(\lambda x . e) = Fv(e) \setminus \{x\}$	$Bv(\lambda x . e) = Bv(e) \cup \{x\}$

Binders

- ▶ $\lambda x . e$
 - ▶ The abstraction operator binds x
 - ▶ The renaming of the x does not change the semantics of the expression
 - ▶ Substitutions do not have effects over bound variables

Free variables (Fv)	Bound variables (Bv)
$Fv(x) = \{x\}$	$Bv(x) = \emptyset$
$Fv(e_1 e_2) = Fv(e_1) \cup Fv(e_2)$	$Bv(e_1 e_2) = Bv(e_1) \cup Bv(e_2)$
$Fv(\lambda x . e) = Fv(e) \setminus \{x\}$	$Bv(\lambda x . e) = Bv(e) \cup \{x\}$

- ▶ Examples: $Fv(\lambda x . x + y) = \{y\}$ and $Bv(\lambda x . x + y) = \{x\}$

Substitution

- ▶ $e_1[e_2/x]$ - the *substitution* of e_2 for the **free** occurrences of x in e_1

Substitution

- ▶ $e_1[e_2/x]$ - the *substitution* of e_2 for the **free** occurrences of x in e_1
- ▶ Definition:

Substitution

- ▶ $e_1[e_2/x]$ - the *substitution* of e_2 for the **free** occurrences of x in e_1
- ▶ Definition:

$$x[e/x] = e$$

Substitution

- ▶ $e_1[e_2/x]$ - the *substitution* of e_2 for the **free** occurrences of x in e_1
- ▶ Definition:

$$x[e/x] = e$$

$$y[e/x] = y$$

when $y \neq x$

Substitution

- ▶ $e_1[e_2/x]$ - the *substitution* of e_2 for the **free** occurrences of x in e_1
- ▶ Definition:

$$\begin{array}{lll} x[e/x] & = & e \\ y[e/x] & = & y \\ (e_1 \ e_2)[e/x] & = & (e_1[e/x] \ e_2[e/x]) \end{array} \quad \text{when } y \neq x$$

Substitution

- ▶ $e_1[e_2/x]$ - the *substitution* of e_2 for the **free** occurrences of x in e_1
- ▶ Definition:

$$\begin{array}{lll} x[e/x] & = & e \\ y[e/x] & = & y \quad \text{when } y \neq x \\ (e_1 \ e_2)[e/x] & = & (e_1[e/x] \ e_2[e/x]) \\ (\lambda y. e')[e/x] & = & \lambda y. e'[e/x] \quad \text{when } x \neq y \wedge y \notin Fv(e) \end{array}$$

Substitution

- ▶ $e_1[e_2/x]$ - the *substitution* of e_2 for the **free** occurrences of x in e_1
- ▶ Definition:

$$\begin{array}{lll} x[e/x] & = & e \\ y[e/x] & = & y \quad \text{when } y \neq x \\ (e_1 e_2)[e/x] & = & (e_1[e/x] e_2[e/x]) \\ (\lambda y.e')[e/x] & = & \lambda y.e'[e/x] \quad \text{when } x \neq y \wedge y \notin Fv(e) \\ (\lambda x.e')[e/x] & = & \lambda x.e' \end{array}$$

Substitution

- ▶ $e_1[e_2/x]$ - the *substitution* of e_2 for the **free** occurrences of x in e_1
- ▶ Definition:

$$\begin{array}{lll} x[e/x] & = & e \\ y[e/x] & = & y \\ (e_1 e_2)[e/x] & = & (e_1[e/x] e_2[e/x]) \\ (\lambda y.e')[e/x] & = & \lambda y.e'[e/x] \\ (\lambda x.e')[e/x] & = & \lambda x.e' \end{array} \quad \begin{array}{l} \text{when } y \neq x \\ \\ \\ \text{when } x \neq y \wedge y \notin Fv(e) \end{array}$$

- ▶ Examples:
 - ▶ $y[1/y] = 1$

Substitution

- ▶ $e_1[e_2/x]$ - the *substitution* of e_2 for the **free** occurrences of x in e_1
- ▶ Definition:

$$\begin{aligned}x[e/x] &= e \\y[e/x] &= y && \text{when } y \neq x \\(e_1 e_2)[e/x] &= (e_1[e/x] e_2[e/x]) \\(\lambda y.e')[e/x] &= \lambda y.e'[e/x] && \text{when } x \neq y \wedge y \notin Fv(e) \\(\lambda x.e')[e/x] &= \lambda x.e'\end{aligned}$$

- ▶ Examples:
 - ▶ $y[1/y] = 1, y[1/x] = y$

Substitution

- ▶ $e_1[e_2/x]$ - the *substitution* of e_2 for the **free** occurrences of x in e_1
- ▶ Definition:

$$\begin{array}{lll} x[e/x] & = & e \\ y[e/x] & = & y \\ (e_1 e_2)[e/x] & = & (e_1[e/x] e_2[e/x]) \\ (\lambda y. e')[e/x] & = & \lambda y. e'[e/x] \\ (\lambda x. e')[e/x] & = & \lambda x. e' \end{array} \quad \begin{array}{l} \text{when } y \neq x \\ \\ \\ \text{when } x \neq y \wedge y \notin Fv(e) \end{array}$$

- ▶ Examples:
 - ▶ $y[1/y] = 1$, $y[1/x] = y$
 - ▶ $((\lambda x. x + y) y)[1/y] =$

Substitution

- ▶ $e_1[e_2/x]$ - the *substitution* of e_2 for the **free** occurrences of x in e_1
- ▶ Definition:

$$\begin{aligned}x[e/x] &= e \\y[e/x] &= y && \text{when } y \neq x \\(e_1 e_2)[e/x] &= (e_1[e/x] e_2[e/x]) \\(\lambda y.e')[e/x] &= \lambda y.e'[e/x] && \text{when } x \neq y \wedge y \notin Fv(e) \\(\lambda x.e')[e/x] &= \lambda x.e'\end{aligned}$$

- ▶ Examples:

- ▶ $y[1/y] = 1, y[1/x] = y$
- ▶ $((\lambda x.x + y) y)[1/y] = ((\lambda x.x + y)[1/y]) (y[1/y]) =$

Substitution

- ▶ $e_1[e_2/x]$ - the *substitution* of e_2 for the **free** occurrences of x in e_1
- ▶ Definition:

$$\begin{aligned}x[e/x] &= e \\y[e/x] &= y && \text{when } y \neq x \\(e_1 e_2)[e/x] &= (e_1[e/x] e_2[e/x]) \\(\lambda y. e')[e/x] &= \lambda y. e'[e/x] && \text{when } x \neq y \wedge y \notin Fv(e) \\(\lambda x. e')[e/x] &= \lambda x. e'\end{aligned}$$

- ▶ Examples:

- ▶ $y[1/y] = 1, y[1/x] = y$
- ▶ $((\lambda x. x + y) y)[1/y] = ((\lambda x. x + y)[1/y]) (y[1/y]) = (\lambda x. (x + y)[1/y]) 1 =$

Substitution

- ▶ $e_1[e_2/x]$ - the *substitution* of e_2 for the **free** occurrences of x in e_1
- ▶ Definition:

$$\begin{array}{lll} x[e/x] & = & e \\ y[e/x] & = & y \\ (e_1 e_2)[e/x] & = & (e_1[e/x] e_2[e/x]) \\ (\lambda y. e')[e/x] & = & \lambda y. e'[e/x] \\ (\lambda x. e')[e/x] & = & \lambda x. e' \end{array} \quad \begin{array}{l} \text{when } y \neq x \\ \\ \\ \text{when } x \neq y \wedge y \notin Fv(e) \end{array}$$

- ▶ Examples:

- ▶ $y[1/y] = 1, y[1/x] = y$
- ▶ $((\lambda x. x + y) y)[1/y] = ((\lambda x. x + y)[1/y]) (y[1/y]) = (\lambda x. (x + y)[1/y]) 1 = (\lambda x. x + 1) 1$

Substitution

- ▶ $e_1[e_2/x]$ - the *substitution* of e_2 for the **free** occurrences of x in e_1
- ▶ Definition:

$$\begin{array}{lll} x[e/x] & = & e \\ y[e/x] & = & y \quad \text{when } y \neq x \\ (e_1 e_2)[e/x] & = & (e_1[e/x] e_2[e/x]) \\ (\lambda y. e')[e/x] & = & \lambda y. e'[e/x] \quad \text{when } x \neq y \wedge y \notin Fv(e) \\ (\lambda x. e')[e/x] & = & \lambda x. e' \end{array}$$

- ▶ Examples:

- ▶ $y[1/y] = 1, y[1/x] = y$
- ▶ $((\lambda x. x + y) y)[1/y] = ((\lambda x. x + y)[1/y]) (y[1/y]) = (\lambda x. (x + y)[1/y]) 1 = (\lambda x. x + 1) 1$
- ▶ $((\lambda x. x + y) y)[1/x] =$

Substitution

- ▶ $e_1[e_2/x]$ - the *substitution* of e_2 for the **free** occurrences of x in e_1
- ▶ Definition:

$$\begin{aligned}x[e/x] &= e \\y[e/x] &= y && \text{when } y \neq x \\(e_1 e_2)[e/x] &= (e_1[e/x] e_2[e/x]) \\(\lambda y. e')[e/x] &= \lambda y. e'[e/x] && \text{when } x \neq y \wedge y \notin Fv(e) \\(\lambda x. e')[e/x] &= \lambda x. e'\end{aligned}$$

- ▶ Examples:

- ▶ $y[1/y] = 1, y[1/x] = y$
- ▶ $((\lambda x. x + y) y)[1/y] = ((\lambda x. x + y)[1/y]) (y[1/y]) = (\lambda x. (x + y)[1/y]) 1 = (\lambda x. x + 1) 1$
- ▶ $((\lambda x. x + y) y)[1/x] = ((\lambda x. x + y)[1/x] (y[1/x])) =$

Substitution

- ▶ $e_1[e_2/x]$ - the *substitution* of e_2 for the **free** occurrences of x in e_1
- ▶ Definition:

$$\begin{array}{lll} x[e/x] & = & e \\ y[e/x] & = & y \quad \text{when } y \neq x \\ (e_1 e_2)[e/x] & = & (e_1[e/x] e_2[e/x]) \\ (\lambda y. e')[e/x] & = & \lambda y. e'[e/x] \quad \text{when } x \neq y \wedge y \notin Fv(e) \\ (\lambda x. e')[e/x] & = & \lambda x. e' \end{array}$$

- ▶ Examples:

- ▶ $y[1/y] = 1, y[1/x] = y$
- ▶ $((\lambda x. x + y) y)[1/y] = ((\lambda x. x + y)[1/y]) (y[1/y]) = (\lambda x. (x + y)[1/y]) 1 = (\lambda x. x + 1) 1$
- ▶ $((\lambda x. x + y) y)[1/x] = ((\lambda x. x + y)[1/x] (y[1/x])) = (? y)$

Alpha - equivalence

- ▶ $\lambda x.x$ and $\lambda y.y$ represent the abstractisation of the *same* function; why?

Alpha - equivalence

- ▶ $\lambda x.x$ and $\lambda y.y$ represent the abstractisation of the *same* function; why?
- ▶ α – *equivalence*:

$\lambda x.e \equiv_{\alpha} \lambda y.e[y/x]$, where y is *fresh* ($\notin Fv(e)$)

Alpha - equivalence

- ▶ $\lambda x.x$ and $\lambda y.y$ represent the abstractisation of the *same* function; why?
- ▶ α – *equivalence*:

$\lambda x.e \equiv_{\alpha} \lambda y.e[y/x]$, where y is *fresh* ($\notin Fv(e)$)

- ▶ Example: $(\lambda x.x + y) \equiv_{\alpha} (\lambda z.(x + y)[z/x]) = (\lambda z.z + y)$

Beta - reduction

- ▶ Computations: done via *rewriting* expressions

Beta - reduction

- ▶ Computations: done via *rewriting* expressions
- ▶ β -reduction:

$$(\lambda x. e) e' \rightarrow_{\beta} e[e'/x]$$

Beta - reduction

- ▶ Computations: done via *rewriting* expressions
- ▶ β -reduction:

$$(\lambda x. e) e' \rightarrow_{\beta} e[e'/x]$$

- ▶ **Redex**: $(\lambda x. e) e'$

Beta - reduction

- ▶ Computations: done via *rewriting* expressions
- ▶ β -reduction:

$$(\lambda x. e) e' \rightarrow_{\beta} e[e'/x]$$

- ▶ **Redex**: $(\lambda x. e) e'$
- ▶ **Reductum** : $e[e'/x]$

Beta - reduction

- ▶ Computations: done via *rewriting* expressions
- ▶ β -reduction:

$$(\lambda x. e) e' \rightarrow_{\beta} e[e'/x]$$

- ▶ **Redex**: $(\lambda x. e) e'$
- ▶ **Reductum** : $e[e'/x]$
- ▶ (Small) examples:
 - ▶ $(\lambda x. x) 1 \rightarrow_{\beta} x[1/x] = 1$

Beta - reduction

- ▶ Computations: done via *rewriting* expressions
- ▶ β -reduction:

$$(\lambda x. e) e' \rightarrow_{\beta} e[e'/x]$$

- ▶ **Redex**: $(\lambda x. e) e'$
- ▶ **Reductum** : $e[e'/x]$
- ▶ (Small) examples:

- ▶ $(\lambda x. x) 1 \rightarrow_{\beta} x[1/x] = 1$
- ▶ $(\lambda x. x + 1) 1 \rightarrow_{\beta} (x + 1)[1/x] = 1 + 1 = 2$

Normal forms

- ▶ When a λ -term does not contain a redex, β -reduction terminates

Normal forms

- ▶ When a λ -term does not contain a redex, β -reduction terminates
- ▶ Such terms are in *normal form*

Normal forms

- ▶ When a λ -term does not contain a redex, β -reduction terminates
- ▶ Such terms are in *normal form*
- ▶ Example: $\lambda x.(\lambda y.x)$ - normal form
- ▶ Example: $\lambda x.((\lambda y.y) x)$ - not in normal form

Normal forms

- ▶ When a λ -term does not contain a redex, β -reduction terminates
- ▶ Such terms are in *normal form*
- ▶ Example: $\lambda x.(\lambda y.x)$ - normal form
- ▶ Example: $\lambda x.((\lambda y.y) x)$ - not in normal form
- ▶ What about: $(\lambda x.x x) (\lambda x.x x)$?

Normal forms

- ▶ When a λ -term does not contain a redex, β -reduction terminates
- ▶ Such terms are in *normal form*
- ▶ Example: $\lambda x.(\lambda y.x)$ - normal form
- ▶ Example: $\lambda x.((\lambda y.y) x)$ - not in normal form
- ▶ What about: $(\lambda x.x x) (\lambda x.x x)$?
- ▶ $(\lambda x.x x) (\lambda x.x x) \rightarrow$

Normal forms

- ▶ When a λ -term does not contain a redex, β -reduction terminates
- ▶ Such terms are in *normal form*
- ▶ Example: $\lambda x.(\lambda y.x)$ - normal form
- ▶ Example: $\lambda x.((\lambda y.y) x)$ - not in normal form
- ▶ What about: $(\lambda x.x x) (\lambda x.x x)$?
- ▶ $(\lambda x.x x) (\lambda x.x x) \rightarrow (x x)[(\lambda x.x x)/x] =$

Normal forms

- ▶ When a λ -term does not contain a redex, β -reduction terminates
- ▶ Such terms are in *normal form*
- ▶ Example: $\lambda x.(\lambda y.x)$ - normal form
- ▶ Example: $\lambda x.((\lambda y.y) x)$ - not in normal form
- ▶ What about: $(\lambda x.x x) (\lambda x.x x)$?
- ▶ $(\lambda x.x x) (\lambda x.x x) \rightarrow (x x)[(\lambda x.x x)/x] =$
 $(\lambda x.x x) (\lambda x.x x)$

Normal forms

- ▶ When a λ -term does not contain a redex, β -reduction terminates
- ▶ Such terms are in *normal form*
- ▶ Example: $\lambda x.(\lambda y.x)$ - normal form
- ▶ Example: $\lambda x.((\lambda y.y) x)$ - not in normal form
- ▶ What about: $(\lambda x.x x) (\lambda x.x x)$?
- ▶ $(\lambda x.x x) (\lambda x.x x) \rightarrow (x x)[(\lambda x.x x)/x] =$
 $(\lambda x.x x) (\lambda x.x x) \rightarrow \dots$

Recursion

- ▶ How to define recursive functions?

Recursion

- ▶ How to define recursive functions?
- ▶ Example:

$$\text{sum} \triangleq \lambda n. \text{if } (n \leq 0) \text{ then } 0 \text{ else } n + (\text{sum}(n - 1))$$

Recursion

- ▶ How to define recursive functions?
- ▶ Example:

$$\text{sum} \triangleq \lambda n. \text{if } (n \leq 0) \text{ then } 0 \text{ else } n + (\text{sum}(n - 1))$$

- ▶ Let's do it in Haskell!

Haskell example explained

► `sum = λ n. if n <= 0 then 0 else n + sum (n - 1)`

```
sum 2 → (λn. if n <= 0 then 0 else n + sum (n - 1)) 2
      → if 2 <= 0 then 0 else 2 + sum (2 - 1)
      → 2 + sum (2 - 1)
      → 2 + sum 1
      → 2 + (λn. if n <= 0 then 0 else n + sum (n - 1)) 1
      → 2 + (if 1 <= 0 then 0 else 1 + sum (1 - 1))
      → 2 + (1 + sum (1 - 1))
      → 2 + (1 + sum 0)
      → 2 + (1 + (λn. if n <= 0 then 0 else n + sum (n - 1)) 0)
      → 2 + (1 + (if 0 <= 0 then 0 else 0 + sum (0 - 1)))
      → 2 + (1 + 0)
      → 2 + 1
      → 3
```

The Y-combinator

- ▶ In theory, λ calculus does not provide “native” support for recursion

The Y-combinator

- ▶ In theory, λ calculus does not provide “native” support for recursion
- ▶ But, it is possible to add it using the Y -combinator

The Y-combinator

- ▶ In theory, λ calculus does not provide “native” support for recursion
- ▶ But, it is possible to add it using the Y -combinator
 - ▶ Combinators are λ expressions without free variables

The Y-combinator

- ▶ In theory, λ calculus does not provide “native” support for recursion
- ▶ But, it is possible to add it using the Y -combinator
 - ▶ Combinators are λ expressions without free variables
- ▶ The Y -combinator:

$$\lambda f.(\lambda x.f (x x)) (\lambda x.f (x x))$$

Evaluation

Haskell example:

```
doubleMe x = x + x
```

```
tripleMe y = 3 * y
```

Evaluation

Haskell example:

```
doubleMe x = x + x
```

```
tripleMe y = 3 * y
```

How to evaluate: `doubleMe (tripleMe 4) ?`

Evaluation

Haskell example:

```
doubleMe x = x + x  
tripleMe y = 3 * y
```

How to evaluate: `doubleMe (tripleMe 4)` ?

Confluence : multiple redexes in expression leads to the same result no matter the evaluation order

Evaluation

Haskell example:

```
doubleMe x = x + x  
tripleMe y = 3 * y
```

How to evaluate: `doubleMe (tripleMe 4)` ?

Confluence : multiple redexes in expression leads to the same result no matter the evaluation order

Strategies:

- ▶ Evaluation by value
- ▶ Evaluation by name
- ▶ Lazy evaluation

Evaluation by value

Steps:

1. Scan expression from left to right and find the first application, say:

`doubleMe (tripleMe 4)`

Evaluation by value

Steps:

1. Scan expression from left to right and find the first application, say:

`doubleMe (tripleMe 4)`

2. Evaluate `doubleMe` :

`(\ x → x + x )`

Evaluation by value

Steps:

1. Scan expression from left to right and find the first application, say:

`doubleMe (tripleMe 4)`

2. Evaluate `doubleMe` :

`(\x → x + x )`

Note: we have obtained a **value** of *functional type*

Evaluation by value

Steps:

1. Scan expression from left to right and find the first application, say:

`doubleMe (tripleMe 4)`

2. Evaluate `doubleMe` :

`(\x → x + x )`

Note: we have obtained a **value** of *functional type*

3. Evaluate `(tripleMe 4)` :

Evaluation by value

Steps:

1. Scan expression from left to right and find the first application, say:

`doubleMe (tripleMe 4)`

2. Evaluate `doubleMe` :

`(\x → x + x )`

Note: we have obtained a **value** of *functional type*

3. Evaluate `(tripleMe 4)` :

`(\y → 3 * y ) 4` $\rightarrow_{\beta}$ `3 * 4` $\rightarrow_{\beta}$ `12`

Evaluation by value

Steps:

1. Scan expression from left to right and find the first application, say:

`doubleMe (tripleMe 4)`

2. Evaluate `doubleMe` :

$(\lambda x \rightarrow x + x)$

Note: we have obtained a **value** of *functional type*

3. Evaluate `(tripleMe 4)` :

$(\lambda y \rightarrow 3 * y) 4 \rightarrow_{\beta} 3 * 4 \rightarrow_{\beta} 12$

Note: we have obtained a **value** of *primitive type*

Evaluation by value

Steps:

1. Scan expression from left to right and find the first application, say:

`doubleMe (tripleMe 4)`

2. Evaluate `doubleMe` :

`(\x → x + x )`

Note: we have obtained a **value** of *functional type*

3. Evaluate `(tripleMe 4)` :

`(\y → 3 * y ) 4 →β 3 * 4 →β 12`

Note: we have obtained a **value** of *primitive type*

4. Reduce the redex using β -reduction:

`(\x → x + x ) 12 →β 12 + 12 →β 24`

Finally, go to step 1

Evaluation by name

Steps:

1. Scan expression from left to right and find the first application, say:

`doubleMe (tripleMe 4)`

Evaluation by name

Steps:

1. Scan expression from left to right and find the first application, say:

`doubleMe (tripleMe 4)`

2. Evaluate `doubleMe` :

`(\x → x + x )`

Evaluation by name

Steps:

1. Scan expression from left to right and find the first application, say:

`doubleMe (tripleMe 4)`

2. Evaluate `doubleMe` :

`(\x → x + x )`

Note: we have obtained a **value** of *functional type*

Evaluation by name

Steps:

1. Scan expression from left to right and find the first application, say:

`doubleMe (tripleMe 4)`

2. Evaluate `doubleMe` :

`(\x → x + x )`

Note: we have obtained a **value** of *functional type*

3. Reduce the redex using β -reduction:

`(\x → x + x ) tripleMe 4` $\rightarrow_{\beta}$

`(tripleMe 4) + (tripleMe 4)` $\rightarrow_{\beta} \cdots \rightarrow_{\beta}$ 24

Finally, go to step 1

Lazy evaluation

- ▶ The call by name's problem: duplicate expressions introduce when performing β -reduction in evaluation by name (step 3).

Lazy evaluation

- ▶ The call by name's problem: duplicate expressions introduce when performing β -reduction in evaluation by name (step 3).

- ▶ Recall:

$(\lambda x \rightarrow x+x) \text{ (tripleMe 4)} \rightarrow_{\beta} (\text{tripleMe 4}) + (\text{tripleMe 4})$

Lazy evaluation

- ▶ The call by name's problem: duplicate expressions introduce when performing β -reduction in evaluation by name (step 3).

- ▶ Recall:

$(\lambda x \rightarrow x+x) \text{ (tripleMe 4)} \rightarrow_{\beta} (\text{tripleMe 4}) + (\text{tripleMe 4})$

Lazy evaluation

- ▶ The call by name's problem: duplicate expressions introduce when performing β -reduction in evaluation by name (step 3).

- ▶ Recall:

$(\lambda x \rightarrow x+x) \text{ (tripleMe 4)} \rightarrow_{\beta} (\text{tripleMe 4}) + (\text{tripleMe 4})$

- ▶ inefficient evaluations introduced!

Lazy evaluation

- ▶ The call by name's problem: duplicate expressions introduce when performing β -reduction in evaluation by name (step 3).

- ▶ Recall:

$(\lambda x \rightarrow x+x) \text{ (tripleMe 4)} \rightarrow_{\beta} (\text{tripleMe 4}) + (\text{tripleMe 4})$

- ▶ inefficient evaluations introduced!

- ▶ **Lazy evaluation**: when (tripleMe 4) is evaluated the first time it keeps a copy and uses it later on

Lazy evaluation

- ▶ The call by name's problem: duplicate expressions introduce when performing β -reduction in evaluation by name (step 3).

- ▶ Recall:

$(\lambda x \rightarrow x+x) \text{ (tripleMe 4)} \rightarrow_{\beta} (\text{tripleMe 4}) + (\text{tripleMe 4})$

- ▶ inefficient evaluations introduced!

- ▶ **Lazy evaluation**: when (tripleMe 4) is evaluated the first time it keeps a copy and uses it later on
- ▶ Evaluation by name + lazy evaluation = *call by need*

Functional programming in Haskell

- ▶ Integers, Booleans
- ▶ Types
- ▶ Lists: head, tail, reverse, reverse, sum, infinite lists
- ▶ Lazy evaluation: examples using functions on infinite lists
- ▶ Higher-order functions: map, foldl
- ▶ Generate prime numbers

Resources

Please refer to the book:

- ▶ **Chapter 11:** http://websrv.dthu.edu.vn/attachments/newsevents/content2415/Programming_Languages_-_Principles_and_Paradigms_thereads1106.pdf

in order to grasp other related items:

1. local environment
2. types
3. pattern matching
4. infinite objects